Programming in C — Complete Semester Notes

Unit 1: Introduction to C

1.1 History & Features

- C was developed by **Dennis Ritchie** at Bell Labs in **1972**.
- It is a **middle-level, procedural, structured** programming language.
- Features: fast execution, portability, rich set of operators, modularity (functions), low-level access via pointers, extensible.

1.2 Structure of a C Program

```c
#include <stdio.h>      // Preprocessor directive
#define PI 3.14         // Macro definition

int add(int, int);      // Function declaration (prototype)

int main() {             // main function - entry point
    int a = 5, b = 10;
    printf("Sum = %d", add(a, b));
    return 0;
}

int add(int x, int y) {  // Function definition
    return x + y;
}
```

Order: Documentation → Preprocessor directives → Global declarations → main() → User-defined functions.

1.3 Tokens

Smallest individual units: **Keywords, Identifiers, Constants, Strings, Operators, Special symbols.**
- **Keywords**: reserved words (32 in C) e.g. `int, float, if, else, for, while, return, struct, void, static, const...` - **Identifiers**: names given to variables/functions (must start with letter/underscore, no spaces, case-sensitive).

1.4 Data Types

Type	Size (typical)	Format Specifier
char	1 byte	%c
int	2/4 bytes	%d
float	4 bytes	%f
double	8 bytes	%lf
short int	2 bytes	%hd
long int	4/8 bytes	%ld
unsigned int	2/4 bytes	%u

Type modifiers: `signed, unsigned, short, long`

1.5 Variables & Constants

- Variable: named memory location whose value can change.
- Constant: fixed value — declared using `const` keyword or `#define`.

```
        const int MAX = 100;
        #define MIN 0
```

Unit 2: Operators & Expressions

2.1 Types of Operators

1. **Arithmetic**: `+ - * / %`
2. **Relational**: `== != > < >= <=`
3. **Logical**: `&& || !`
4. **Assignment**: `= += -= *= /= %=`
5. **Increment/Decrement**: `++ --` (pre and post)
6. **Bitwise**: `& | ^ ~ << >>`
7. **Conditional (Ternary)**: `condition ? expr1 : expr2`
8. **Comma operator**: `,`
9. **sizeof operator**: returns size in bytes

2.2 Operator Precedence (High → Low, common exam Q)

```
() [] ->        (highest)
! ~ ++ -- (unary) sizeof
* / %
+ -
<< >>
< <= > >=
== !=
```

```
&
^
|
&&
||
?:
= += -= ...      (lowest)
```

2.3 Type Conversion

- **Implicit (automatic)**: compiler converts lower to higher data type (int → float).
- **Explicit (type casting)**: `(float) a / b;`

Unit 3: Control Statements

3.1 Decision Making

```c
// if-else
if (a > b) { ... } else { ... }

// else-if ladder
if (x > 0) ...
else if (x == 0) ...
else ...

// switch-case
switch (choice) {
    case 1: printf("One"); break;
    case 2: printf("Two"); break;
    default: printf("Invalid");
}
```

3.2 Loops

```c
// for loop
for (i = 0; i < n; i++) { ... }

// while loop
while (condition) { ... }

// do-while loop (executes at least once)
do { ... } while (condition);
```

3.3 Jump Statements

- `break` — exits loop/switch immediately.
- `continue` — skips current iteration, moves to next.
- `goto` — unconditional jump to a labeled statement.
- `return` — exits function, optionally returning a value.

Unit 4: Arrays & Strings

4.1 Arrays

- Collection of similar data types stored in contiguous memory.

```
int arr[5] = {1, 2, 3, 4, 5};      // 1-D array
int mat[2][3] = {{1,2,3},{4,5,6}}; // 2-D array
```

- Array name acts as a **pointer to the first element**.
- Accessing: `arr[i]` is equivalent to `*(arr + i)`.

4.2 Strings

- A string is a **character array terminated by** `\0` (null character).

```
char name[20] = "Hello";
```

Common String Functions (`<string.h>`): | Function | Use | |—|—| | strlen(s) | length of string | | strcpy(d,s) | copy s into d | | strcat(d,s) | concatenate s to d | | strcmp(s1,s2) | compare (0 if equal) | | strrev(s) | reverse string | | strlwr / strupr | case conversion |

Unit 5: Functions

5.1 Basics

```
return_type function_name(parameter_list) {
    // body
    return value;
}
```

- **Function prototype/declaration**, **definition**, and **call**.
- **Actual parameters** (in call) vs **Formal parameters** (in definition).

5.2 Parameter Passing

- **Call by Value**: copy of variable passed; original unchanged.
- **Call by Reference**: address passed; original CAN be modified (via pointers in C).

5.3 Types of Functions

- Library (predefined) functions vs User-defined functions.
- Recursive function — a function that calls itself.

```
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

5.4 Storage Classes

Class	Scope	Lifetime	Default value
auto	local	function block	garbage

static	local/global	entire program	0
extern	global	entire program	0
register	local	function block	garbage

Unit 6: Pointers

6.1 Basics

- A pointer stores the **address** of another variable.

```
int a = 10;
int *p = &a;     // p holds address of a
printf("%d", *p); // dereferencing - prints value of a
```

6.2 Pointer Arithmetic

- `p+1` moves pointer forward by `sizeof(data type)` bytes.
- Valid operations: increment, decrement, addition/subtraction of integer, comparison.

6.3 Types of Pointers

- **Null pointer** (`NULL`), **Void pointer** (generic), **Wild pointer** (uninitialized), **Dangling pointer** (points to freed memory).

6.4 Pointers & Arrays / Functions

```
int arr[5];
int *p = arr;        // array decays to pointer

void update(int *x) { *x = 100; }  // call by reference
```

6.5 Pointer to Pointer & Function Pointers

```
int a = 5, *p = &a, **pp = &p;
int (*funcPtr)(int, int) = add;   // pointer to function
```

Unit 7: Structures & Unions

7.1 Structure

- Groups **different data types** under one name.

```
struct Student {
    char name[20];
    int roll;
    float marks;
};
```

```
struct Student s1 = {"Aman", 1, 89.5};
printf("%s", s1.name);
```

- **Array of structures**, **Nested structures**, **Structure pointers** (`s1.name` vs `ptr->name`).

7.2 Union

- All members **share the same memory location**; size = size of largest member.

```
union Data {
    int i;
    float f;
    char c;
};
```

Struct vs Union (important exam question): | Structure | Union | |—|—| | Separate memory for each member | Shared memory for all members | | Size = sum of all members | Size = size of largest member | | All members can hold values simultaneously | Only one member holds a valid value at a time |

7.3 typedef & enum

```
typedef struct Student STU;
enum Day {MON, TUE, WED};   // MON=0, TUE=1, WED=2
```

Unit 8: Dynamic Memory Allocation

Functions from `<stdlib.h>` : | Function | Purpose | |—|—| | malloc(size) | allocates uninitialized memory block | | calloc(n, size) | allocates and zero-initializes n blocks | | realloc(ptr, newsize) | resizes previously allocated memory | | free(ptr) | deallocates memory |

```
int *p = (int*) malloc(5 * sizeof(int));
if (p == NULL) printf("Allocation failed");
free(p);
```

Unit 9: File Handling

9.1 File Modes

Mode	Meaning
“r”	read
“w”	write (overwrite)
“a”	append
“r+”	read & write
“w+”	read & write (overwrite)

“a+”	read & append

9.2 Common Operations

```
        FILE *fp = fopen("data.txt", "w");
        if (fp == NULL) { printf("Error"); exit(1); }

        fprintf(fp, "Hello %d", 5);   // write formatted
        fscanf(fp, "%d", &x);          // read formatted
        fputc('A', fp); fgetc(fp);     // char I/O
        fputs("Hello", fp); fgets(buf, 100, fp);  // string I/O

        fclose(fp);
```

- `feof(fp)` — checks end of file.

Unit 10: Preprocessor Directives

```
        #include <stdio.h>       // header inclusion
        #define SIZE 10          // macro (object-like)
        #define SQUARE(x) ((x)*(x))  // macro (function-like)
        #ifdef DEBUG ... #endif  // conditional compilation
        #undef SIZE
```

Preprocessing happens **before compilation** — pure text substitution, no type checking.

Frequently Asked Theory Questions (Quick Answers)

Q1. Difference between `while` and `do-while`? `while` checks condition first (may run 0 times); `do-while` executes body first, then checks (runs at least once).

Q2. Difference between call by value and call by reference? Call by value passes a copy (changes don’t reflect back); call by reference passes address (changes reflect in original).

Q3. Difference between `malloc()` and `calloc()`? `malloc` allocates a single block without initialization; `calloc` allocates multiple blocks and initializes them to zero.

Q4. What is a pointer? A variable that stores the memory address of another variable.

Q5. Difference between array and pointer? Array is a fixed-size collection with allocated memory; pointer is a variable that can point to any address and be reassigned.

Q6. What is recursion? Give example. A function calling itself to solve a smaller instance of the same problem — e.g., factorial, Fibonacci.

Q7. Difference between structure and union? See table in Unit 7.2 above.

Q8. What is dangling pointer? A pointer that still points to a memory location after that memory has been freed.

Q9. What is the difference between `break` and `continue`? `break` exits the loop entirely; `continue` skips the current iteration and continues with the next.

Q10. What is a NULL pointer vs a wild pointer? NULL pointer explicitly points to nothing (`(void*)0`); wild pointer is uninitialized and points to a random/unknown location.

Important Programs to Practice (Common in Exams)

1. Swap two numbers (with and without third variable, using pointers)
2. Factorial using recursion & loop
3. Fibonacci series
4. Prime number check
5. Palindrome check (number & string)
6. Bubble sort / Selection sort array
7. Matrix addition & multiplication
8. String reversal without using library function
9. Largest/smallest element in array
10. Structure to store and display student records
11. File handling — write & read from a file
12. Linked list creation and traversal (using structures & pointers — often part of advanced C syllabus)

Tip for exam: Practice writing these programs by hand, memorize the syntax of loops/functions/pointers precisely, and be ready to explain the difference-based theory questions (Q1–Q10) — these are asked almost every semester.